Principal component analysis

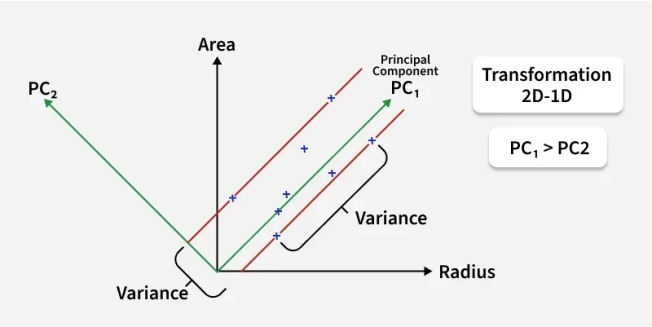
From rotated point clouds to handwritten digits

A six-page guide with two executable notebooks

1. Replace many measurements with fewer coordinates

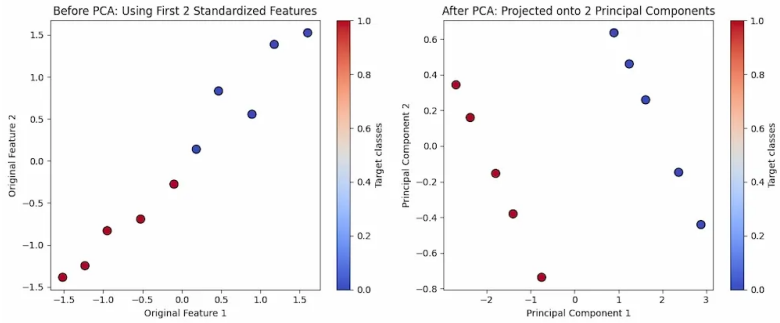
A row may contain two coordinates of a point or 64 pixel measurements of an image. **Principal component analysis (PCA)** learns new orthogonal directions, then represents each observation by its coordinates along a chosen subset of them. It is an unsupervised transformation: it can operate without class labels.

Reading connection. GeeksforGeeks presents a sequence of feature preparation, covariance calculation, eigenvector ranking, and projection onto leading components [1]. Its illustrations below connect an original feature space with a reduced coordinate system.



A 2D-to-1D projection diagram from the source article.

Reproduced from GeeksforGeeks [1], image shown under "Pick the Top Directions & Transform Data." The diagram's PC labels refer to axes; variance is the quantity used to rank them.



Source illustration: two original standardized features versus two PCA scores.

Reproduced from GeeksforGeeks [1], original caption "PCA Algorithm." These are the source article's toy examples, separate from this supplement's experiments. Images are converted from WebP to PNG and resized. Rights remain with their respective rights holders; see ATTRIBUTION.md.

What PCA provides. A *direction* describes a combination of original features; a *score* gives one row's coordinate along it. Keeping all directions rotates the centered data without losing information. Keeping fewer directions discards part of the geometry.

Learning route. Run `01_geometric_projection.ipynb` to see directions and reconstruction; then `02_handwritten_digits.ipynb` for real image compression and a separate classification task. Both contain saved outputs and exercises with runnable reference solutions. No data downloads are needed.

Key idea. PCA preserves variance under the chosen feature geometry. High retained variance does not guarantee class separation, scientific relevance, or a particular predictive accuracy.

2. The mathematics: center, rotate, project

Let $X \in \mathbb{R}^{n\times p}$ contain one observation per row. Compute the column mean $\bar{x}$ and centered matrix $X_c = X - \mathbf{1}\bar{x}^T$. The sample covariance is

$$S = \frac{X_c^T X_c}{n-1}.$$

For a unit direction v, the scores are $t = X_c v$ and their sample variance is $v^T S v$. The first direction solves $\max_{\|v\|=1} v^T S v$. Differentiating the constrained objective gives $2Sv - 2\lambda v = 0$, so $Sv = \lambda v$. Choose a leading eigenvector; later directions are orthogonal eigenvectors ranked by descending eigenvalues $\lambda_1 \geq \cdots \geq \lambda_p \geq 0$.

Coordinates and reconstruction. Write $V_q = [v_1, \ldots, v_q] \in \mathbb{R}^{p\times q}$. Then

$$\boxed{T_q = X_c V_q, \qquad \widehat{X}_q = \mathbf{1}\bar{x}^T + T_q V_q^T.}$$

T_q has one row per observation and q columns. Reconstruction returns to the original p features, replacing omitted principal coordinates by zero after centering.

An exact hand calculation. Take rows $(-2,-2), (-1,-1), (1,1), (2,2)$. Their mean is $(0,0)$ and

$$S = \frac{1}{3}\begin{pmatrix} 10 & 10 \\ 10 & 10 \end{pmatrix}, \qquad v_1 = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad \lambda_1 = \frac{20}{3}, \quad \lambda_2 = 0.$$

PC1 scores are $(-2\sqrt{2}, -\sqrt{2}, \sqrt{2}, 2\sqrt{2})$. One component reconstructs every row exactly: the data lie on a line. A noisy cloud, as on page 3, leaves perpendicular residuals.

How much is retained? For positive total variance,

$$\mathrm{PVE}_j = \frac{\lambda_j}{\sum_\ell \lambda_\ell}, \qquad \mathrm{CPVE}(q) = \frac{\sum_{j=1}^q \lambda_j}{\sum_\ell \lambda_\ell}.$$

PVE is proportion of variance explained; CPVE is its cumulative value. Within the fitting data, the optimal orthogonal rank-q reconstruction satisfies

$$\boxed{\|X - \widehat{X}_q\|_F^2 = (n-1)\sum_{j>q} \lambda_j.}$$

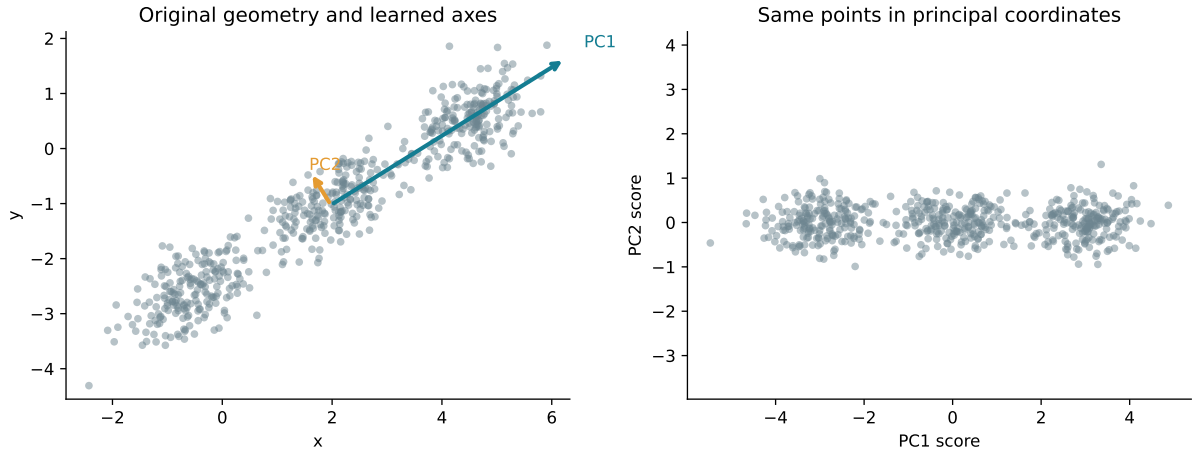
The squared Frobenius norm sums squared errors over every row and feature. Divide by np for mean squared error (MSE) per scalar entry. Test-set error must be measured separately.

SVD implementation [2,3,8]. If $X_c = UDV^T$, singular values d_j give $\lambda_j = d_j^2/(n-1)$ and $T_q = U_q D_q$. Working directly with X_c avoids explicitly forming the covariance matrix. scikit-learn's `components_` stores directions as rows, or V_q^T.

Key idea. The fitting scores are uncorrelated. They need not be statistically independent. Changing both a direction's sign and its scores' signs preserves every reconstructed row.

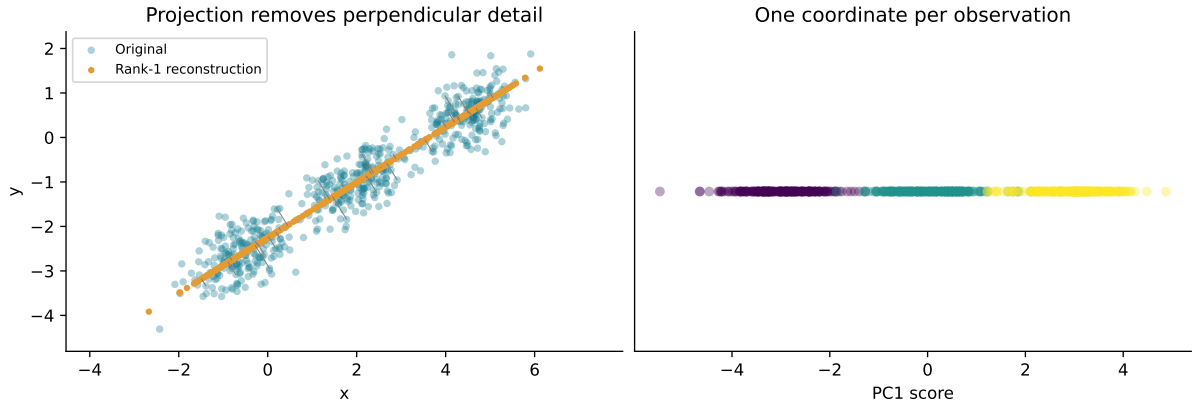
3. Geometric blobs: see what one component removes

Notebook 1 generates three blobs along a common axis, rotates them by 32°, and translates them. Both coordinates share units. PCA sees the coordinates only; simulated blob membership is used to color a later diagnostic.



Original plot: notebook 1. Arrows extend two component SDs from the mean; direction vectors are unit length.

Reduce and reconstruct. Keeping PC1 retains 98.12% of fitting variance. The reconstruction MSE is 0.0603 per original scalar entry. The residual is perpendicular to the retained direction. The notebook checks this, verifies the eigenvalue/error identity, and compares covariance eigenvectors with scikit-learn's SVD result.



Original plot: notebook 1. Short segments show selected residuals. The right-hand strip contains one numerical coordinate per observation; colors indicate generating blobs for an audit.

Centering versus standardization [4]. Centering subtracts a mean. Standardization also divides by a column's SD. PCA in scikit-learn centers automatically and does not standardize. With different units, decide whether absolute variation or comparable per-feature variance should define distance. A unit change can alter the principal directions. With constant columns, standardization cannot create genuine variation. Outliers can strongly influence both means and covariance.

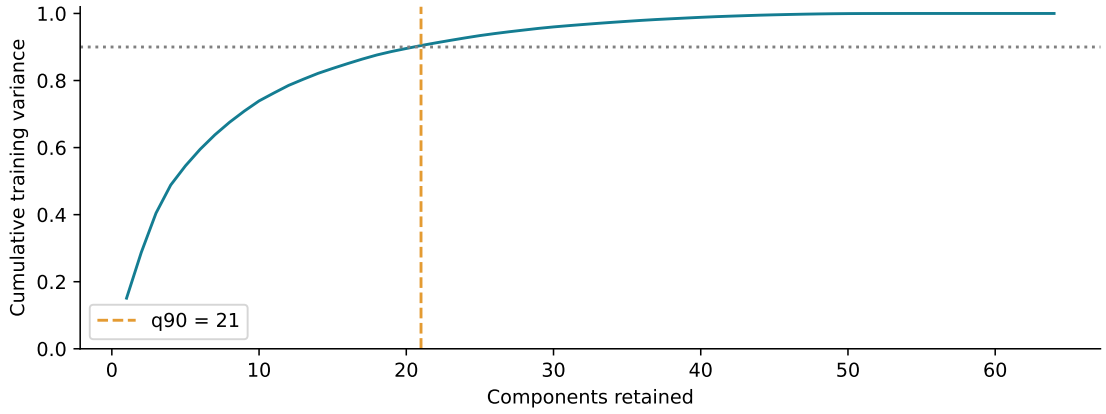
```
pca = PCA(n_components=1, svd_solver="full")
scores = pca.fit_transform(X)        # shape: (n, 1)
reconstruction = pca.inverse_transform(scores)  # (n, 2)
new_scores = pca.transform(X_new)    # use the fitted mean/axes
```

Read the API [2]. `mean_` stores the fitting mean; `components_` stores directions; `explained_variance_` stores score variances using divisor $n - 1$; `explained_variance_ratio_` stores their fractions of fitting variance. `transform` returns coordinates, not class predictions or cluster IDs.

4. Real images: choose the dimension from training data

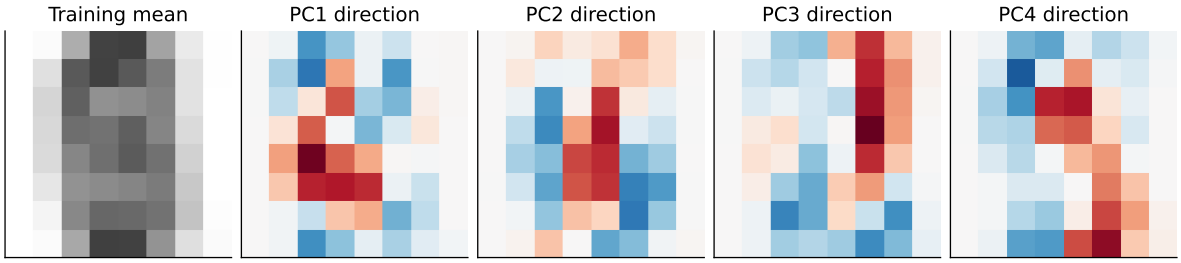
Notebook 2 uses scikit-learn's **Digits** collection: 1,797 handwritten images, each an 8×8 grid flattened to 64 measurements [5,6]. These are counts on a shared 0–16 scale. We retain this common scale so squared error has a direct pixel-feature interpretation. PCA centers without standardizing each pixel separately.

Split first [7]. A fixed stratified 75/25 split gives 1347 training and 450 test images. Learn the mean and full PCA basis from training rows only. Set a 90% variance threshold in advance and choose the smallest q that meets it on training data. The result is $q = 21$, retaining 90.43% of training variance. Digit labels do not enter PCA fitting or this dimension rule.



Original experiment: notebook 2. The dashed vertical line marks the training-selected dimension.

Interpret a component as a pattern. Each direction has 64 signed coefficients. Reshape them into an 8×8 image: a row's score scales that pattern before it is added to the mean image. These directions are variations around the mean, not digit templates or class probabilities.



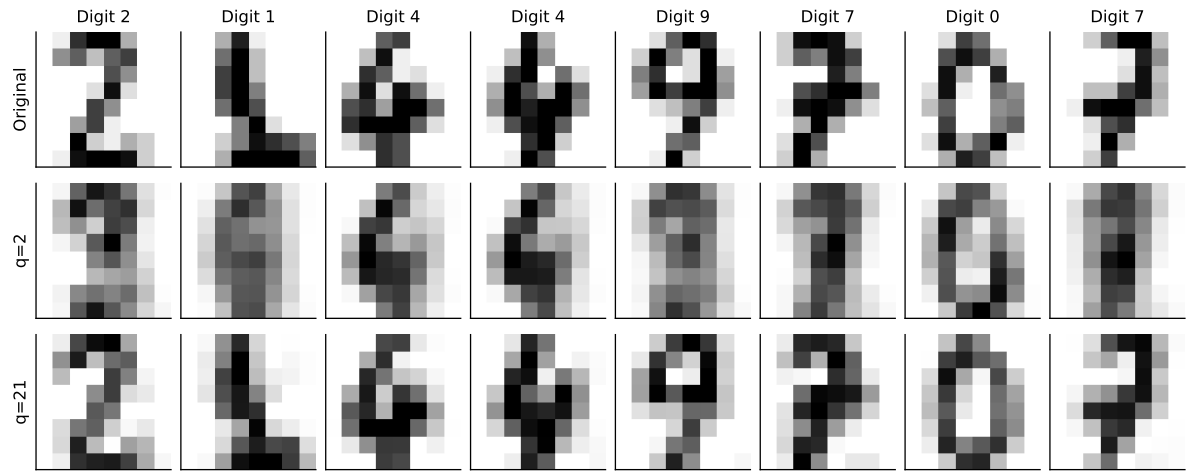
Original plot using UCI-derived images [5,6]. All four directions share a symmetric coefficient color scale. The mean uses the original pixel scale. A component's overall sign is arbitrary.

Inspect the tradeoff. For each candidate dimension, reconstruct rows with the same learned basis and report training and test MSE. More components reduce orthogonal reconstruction error, but also require more scores. The score count alone omits the stored basis and mean. Floating-point scores and small integer pixel counts also have different storage costs.

Key idea. A 2D score plot represents only 28.7% of training variance here. Use it as a partial view; the 90% representation uses 21 coordinates.

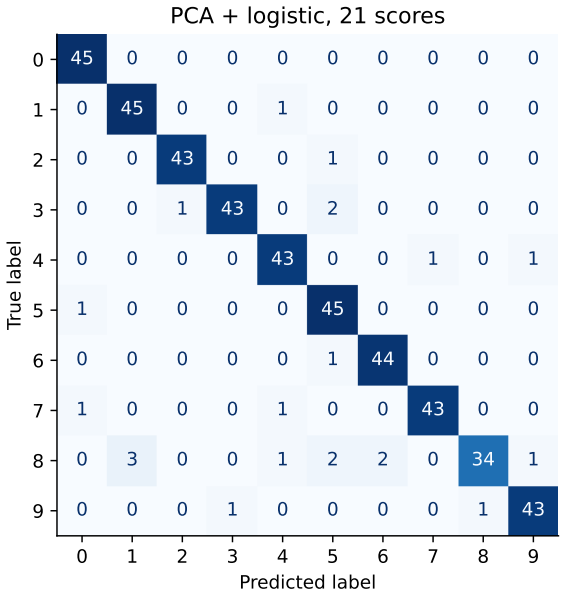
5. Reconstruction and classification answer different questions

Project held-out rows with the training mean and directions. The image grid uses the first eight test rows in split order, avoiding selection of particularly attractive reconstructions. At $q = 21$, held-out MSE is 1.946 per pixel feature. Metrics use unmodified reconstructed values; all displayed images share a 0–16 range.



Original reconstructions of UCI-derived test images [5,6], notebook 2. The 2D representation removes much more detail than the training-selected 90% representation.

PCA does not supply a digit label. To predict a digit, train a separate logistic regression model on the retained scores. A second fixed model uses all 64 centered features as a baseline. Both use $C = 1$ and an iteration limit of 3,000; there is no test-driven tuning [9].



Held-out classification results

Representation	Accuracy
64 centered features	97.33%
21 PCA scores	95.11%

The confusion matrix belongs to the PCA-plus-logistic pipeline. Rows are true digit labels; columns are predictions. These are observed results on one fixed split.

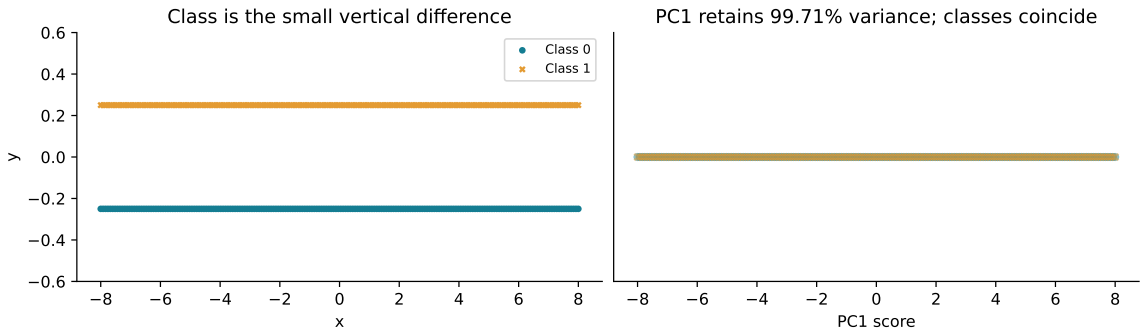
Original comparison, notebook 2. Each pipeline learns its transformations on training images only.

What the comparison supports. It measures how this chosen reduction changes a particular classifier's held-out predictions. Retaining 90% variance does not promise 90% accuracy, and fewer coordinates do not by themselves prove faster end-to-end execution. The example does not tune or benchmark runtime.

Scope of evaluation. The packaged collection is the source dataset's 1,797-image test portion; our split subdivides that collection. It is a random image split, not a split by writer. Results therefore do not establish performance on unseen writers or on other image sources.

6. Limits, exercises, and sources

Variance can miss the task. Two horizontal bands can differ only in a small vertical offset. PC1 keeps the large horizontal variation and maps paired opposite-class points to identical scores. In notebook 1 this retains 99.71% of variance while losing the band label.



Original counterexample: notebook 1. Colors and marker shapes identify the classes only for this audit.

Other limits. Linear directions do not automatically unfold curved structure. Small-variance components can contain real signal, so truncation is not guaranteed denoising. Outliers can rotate the axes. Similar eigenvalues can make individual directions unstable even when their shared subspace is stable. Whitening is a further rescaling of scores, rather than ordinary rotation.

Exercises. Runnable reference solutions appear in both notebooks.

1. Flip a direction and its scores together; verify unchanged reconstruction.
2. Multiply one geometric coordinate by 100; compare raw and standardized PCA.
3. Find the smallest dimension retaining 95% of training digit variance; report training MSE.
4. Count stored scores, direction coefficients, and mean entries. Explain why this is not a byte-compression ratio.

References and attribution

Accessed 27 October 2026. Page 1 includes a short paraphrase and two reproduced images from [1]. The remaining prose, derivations, code, experiments, and plots were prepared for this supplement using the technical references below. Direct image URLs and checksums are in `ATTRIBUTION.md`.

[1] GeeksforGeeks. Principal Component Analysis (PCA). Image rights remain with their respective rights holders; no open image license is asserted.

[2] scikit-learn. PCA API reference.

[3] scikit-learn. Decomposition guide: PCA.

[4] scikit-learn. StandardScaler API reference.

[5] scikit-learn. load_digits API and packaged dataset description.

[6] Alpaydin, E. & Kaynak, C. (1998). *Optical Recognition of Handwritten Digits* [Dataset]. UCI Machine Learning Repository. doi:10.24432/C50P49. CC BY 4.0. We use the scikit-learn subset; PCA projections and reconstructions are analysis transformations.

[7] scikit-learn. Common pitfalls: data leakage.

[8] NumPy. Singular value decomposition and symmetric-matrix eigenvectors.

[9] scikit-learn. LogisticRegression API reference.